PHY4605 Computational Methods in Physics
Learning Note | Week 8

Ordinary Differential Equations: Simulate a changing state from an initial condition

Turn a familiar rate law into small, visible temperature updates, then test how timestep size changes the simulation.

Physical question. A hot object cools toward room temperature. Starting at 80 °C in a 20 °C room, what temperature does an Euler simulation predict from 0 to 300 s, and how does the timestep affect confidence in that prediction?

Learning outcomes

1. identify the state, rate, initial condition, timestep, and final time in a first-order physical model;
2. trace and complete an Euler update loop that produces a labelled temperature trajectory; and
3. validate the simulation with smaller timesteps, an exact reference, and physical initial-value and trend checks.

How to use this note

Read the physical question and rate law first. Pause at each prediction checkpoint, trace the first updates by hand, then run the starter code and compare the trajectory with the stated validation evidence.

1. A changing state needs a starting value

The **state variable** is the object's uniform temperature, $T(t)$. Its value changes with time, so one number is not enough: we need a sequence of temperatures. A **first-order model** uses the first time derivative to describe how one state changes. In plain language, it gives the current rate of change, and one starting value selects the trajectory. Newton cooling gives the rate law

$$\frac{dT}{dt} = -\frac{T - T_{\text{env}}}{\tau}.$$

Here $T_{\text{env}} = 20\,^{\circ}\text{C}$ is the constant room temperature and $\tau = 100\,\text{s}$ is a cooling timescale. The initial condition fixes the first state:

$$T(0) = T_0 = 80\,^{\circ}\text{C}, \qquad 0 \le t \le 300\,\text{s}.$$

The derivative dT/dt is the **rate of change of temperature**, in $^{\circ}\text{C}\,\text{s}^{-1}$. At $t = 0$,

$$\left.\frac{dT}{dt}\right|_{t=0} = -\frac{80 - 20}{100} = -0.6\,^{\circ}\text{C}\,\text{s}^{-1}.$$

The negative sign matches the physical prediction: an object warmer than its surroundings cools. The rate becomes smaller in magnitude as T approaches T_{env}.

Prediction checkpoint. For this model, every simulated temperature should begin at 80 °C, decrease toward 20 °C, and remain between these values. A result that rises or falls below the room temperature needs investigation.

2. Euler uses the current rate for one short step

Choose a timestep h, measured in seconds. Euler's method replaces the continuous change during one timestep by the current rate multiplied by the timestep:

$$T_{n+1} = T_n + h\left(-\frac{T_n - T_{\text{env}}}{\tau}\right).$$

Read this from left to right: start with the current temperature T_n, calculate its cooling rate, then add *rate* $\times$ *timestep* to obtain the next temperature T_{n+1}. The product has the correct unit:

$$^\circ\text{C}\,\text{s}^{-1} \times \text{s} = {}^\circ\text{C}.$$

The method needs a sequence of times, a first temperature from the initial condition, and one update for each adjacent pair of times. It does not solve the differential equation symbolically; it constructs a numerical approximation step by step.

3. Trace the first two Euler updates

Use the deliberately coarse starting timestep $h = 50\,\text{s}$. The first rate is $-0.6\,^\circ\text{C}\,\text{s}^{-1}$, so

$$T_1 = 80 + 50(-0.6) = 50\,^\circ\text{C}.$$

At $t = 50\,\text{s}$ the object is still warmer than the room. Its new rate is

$$-\frac{50 - 20}{100} = -0.3\,^\circ\text{C}\,\text{s}^{-1},$$

so the second update is

$$T_2 = 50 + 50(-0.3) = 35\,^\circ\text{C}.$$

step n	t_n (s)	T_n (°C)	rate at T_n (°C s^{-1})
0	0	80	−0.60
1	50	50	−0.30
2	100	35	−0.15

A common model or logic defect. In the update, use the *current* value T_n on the right-hand side. Reusing T_0 at every step would apply the original cooling rate repeatedly, even after the object has cooled. MATLAB can run that code, but the physics would be wrong.

4. Pseudocode before MATLAB

1. state the environment temperature, initial temperature, timescale, timestep, and final time with units;
2. create all time positions from 0 to the final time using the chosen timestep;
3. reserve one temperature value for every time position, then place the initial condition in the first position;
4. for each current position, calculate the cooling rate from the current temperature;
5. use Euler's update rule to store the next temperature;
6. plot temperature against time with axis labels and units; and
7. check the initial value and trend, then compare the result with a smaller timestep and a reference.

5. An Euler update with starter code

The code below is starter code. The loop counter `n` labels the *current* position. MATLAB uses one-based indexing, so the current temperature is `T_C(n)` and the next position is `T_C(n+1)`.

Listing 1: Euler simulation of Newton cooling with a labelled temperature plot.

```
T_room_C = 20;
T_initial_C = 80;
tau_s = 100;
dt_s = 50;
t_final_s = 300;

nSteps = t_final_s / dt_s;
t_s = (0:nSteps) * dt_s;
T_C = zeros(size(t_s));
T_C(1) = T_initial_C;       % initial condition at t = 0

for n = 1:nSteps
    rate_C_per_s = -(T_C(n) - T_room_C) / tau_s;
    T_C(n+1) = T_C(n) + dt_s * rate_C_per_s;
end

plot(t_s,T_C,'o-','LineWidth',1.2)
xlabel('Time, t (s)')
ylabel('Temperature, T (degrees Celsius)')
grid on
```

Before running, trace the first two loop passes against the table above. Then change only `dt_s` to 25 s or 12.5 s and predict what will happen to the plotted points and the final temperature.

6. Timestep comparison and an independent reference

For this teaching model only, a supplied exact solution provides an independent reference:

$$T_{\text{exact}}(t) = 20 + 60e^{-t/100}.$$

It gives $T_{\text{exact}}(300\,\text{s}) = 22.9872\,^{\circ}\text{C}$. The figure compares this continuous reference with Euler simulations. *Accessibility description: a black smooth cooling curve falls from* 80 °C *toward the dashed* 20 °C *room-temperature line. Euler points using smaller timesteps lie progressively closer to the black curve.*

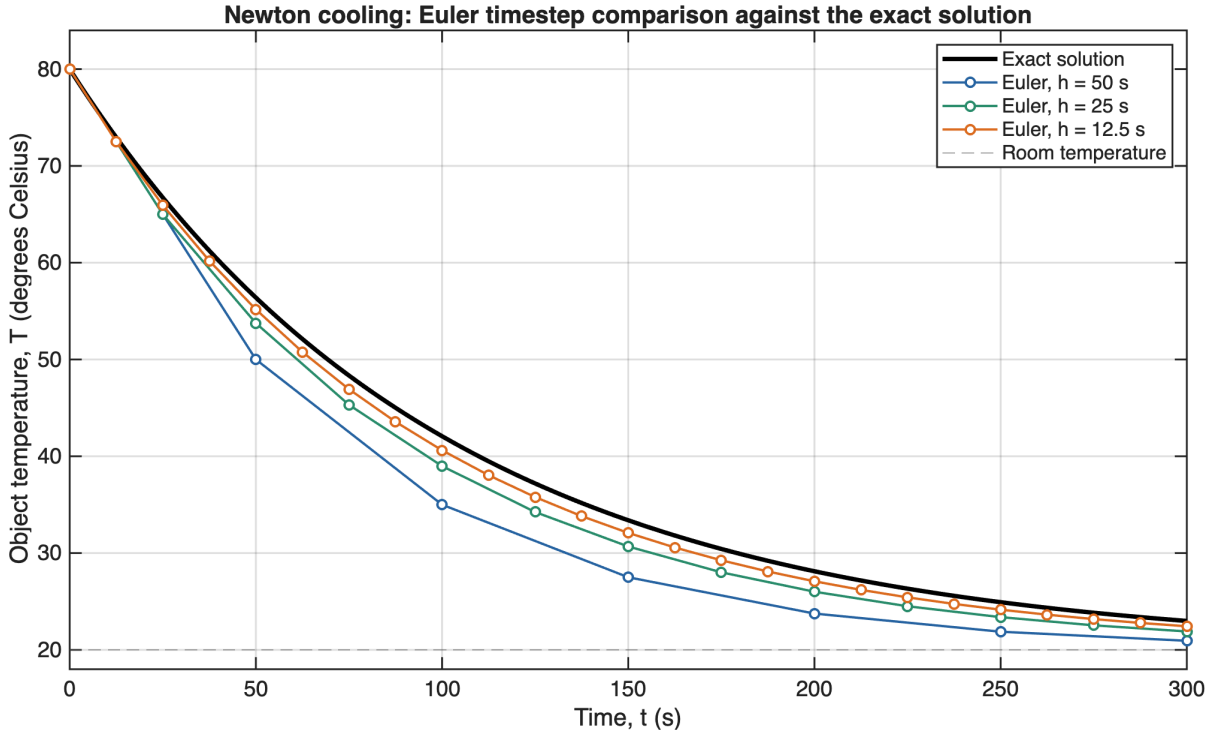


Figure 1: Newton cooling from 80 °C toward a 20 °C environment. Euler trajectories use $h = 50$, 25, and 12.5 s; the black line is the supplied exact reference.

h (s)	Euler $T(300\,\text{s})$ (°C)	absolute error (°C)	comparison
50.0	20.9375	2.0497	coarsest supplied result
25.0	21.9006	1.0866	closer to reference
12.5	22.4341	0.5531	closest supplied result

The physics and final time are unchanged across the rows. Only h changes. For these supplied steps, smaller timesteps give endpoint values closer to the reference. This is evidence about this model and interval, rather than a guarantee that every timestep is acceptable for every ODE.

7. Required validation: check the start, the trend, and the reference

The Euler trajectory with $h = 50\,\text{s}$ starts at the required 80 °C, and its first two calculated values are 50 °C and 35 °C, as traced by hand. Every plotted trajectory decreases toward the 20 °C environment and remains within the physical bounds for this case.

Validation statement. At the same final time, the endpoint error decreases from 2.0497 °C for $h = 50\,\text{s}$ to 0.5531 °C for $h = 12.5\,\text{s}$. Together with the correct initial value and cooling trend, this supports the finer Euler result for the stated Newton-cooling model.

8. Interpret the simulated trajectory

Temperature falls quickly at first because the difference $T - T_{\text{env}}$ is large. As the object approaches the room temperature, the magnitude of the rate decreases, so the curve flattens. At 300 s, the exact model predicts about 23.0 °C: the object is close to, but has not reached, the environment temperature.

The numerical result is only as appropriate as its physical model. Here we assumed a constant environment temperature and one constant cooling timescale. Changing airflow, contact with another object, or internal heat generation could make that assumption unsuitable.

Guided extension: supplied `ode45` reference

MATLAB's `ode45` is a solver that can produce a reference trajectory without writing the Euler loop. Read and interpret the supplied call as reference evidence. You are not expected to implement `ode45` independently; this week's main task remains the visible Euler update and its validation:

Listing 2: Reading a supplied ode45 reference trajectory for the cooling model.

```
coolingRate = @(t,T) -(T - T_room_C) / tau_s;
[t45_s,T45_C] = ode45(coolingRate,[0 t_final_s],T_initial_C);
plot(t45_s,T45_C,'k-')
```

The anonymous function states the same rate law. The interval `[0 t_final_s]` and initial condition `T_initial_C` remain physical inputs that must be checked.

Optional extension

Optional extensions include deriving formal Euler stability conditions, comparing Runge-Kutta methods, simulating a stiff system, handling events, or coupling several nonlinear ODEs. They are not required for this week's pathway.

Three takeaways

1. A first-order ODE simulation needs a state, a rate law, an initial condition, a timestep, and a final time with units.
2. Euler updates the next state from the current state and current rate; trace the first steps before trusting a loop.
3. Compare timesteps with an independent reference and test the initial value and physical trend before interpreting a numerical trajectory.

Preparation for the practical. For each supplied ODE context, identify the state and rate, label the initial condition and units, trace two Euler steps, then record an initial-value/trend check plus a timestep or reference comparison before explaining the output.